ARQUITECTURA Y SISTEMAS OPERATIVOS

CLASE 3

Procesos

REPASO

Repaso de la clase anterior

En la clase anterior vimos que el sistema operativo:

- Actúa como intermediario entre hardware y software
- Administra recursos
- Ofrece servicios a los programas
- Toma control del sistema cuando ocurre un evento relevante

Antes de estudiar procesos, vamos a recordar dos mecanismos clave:

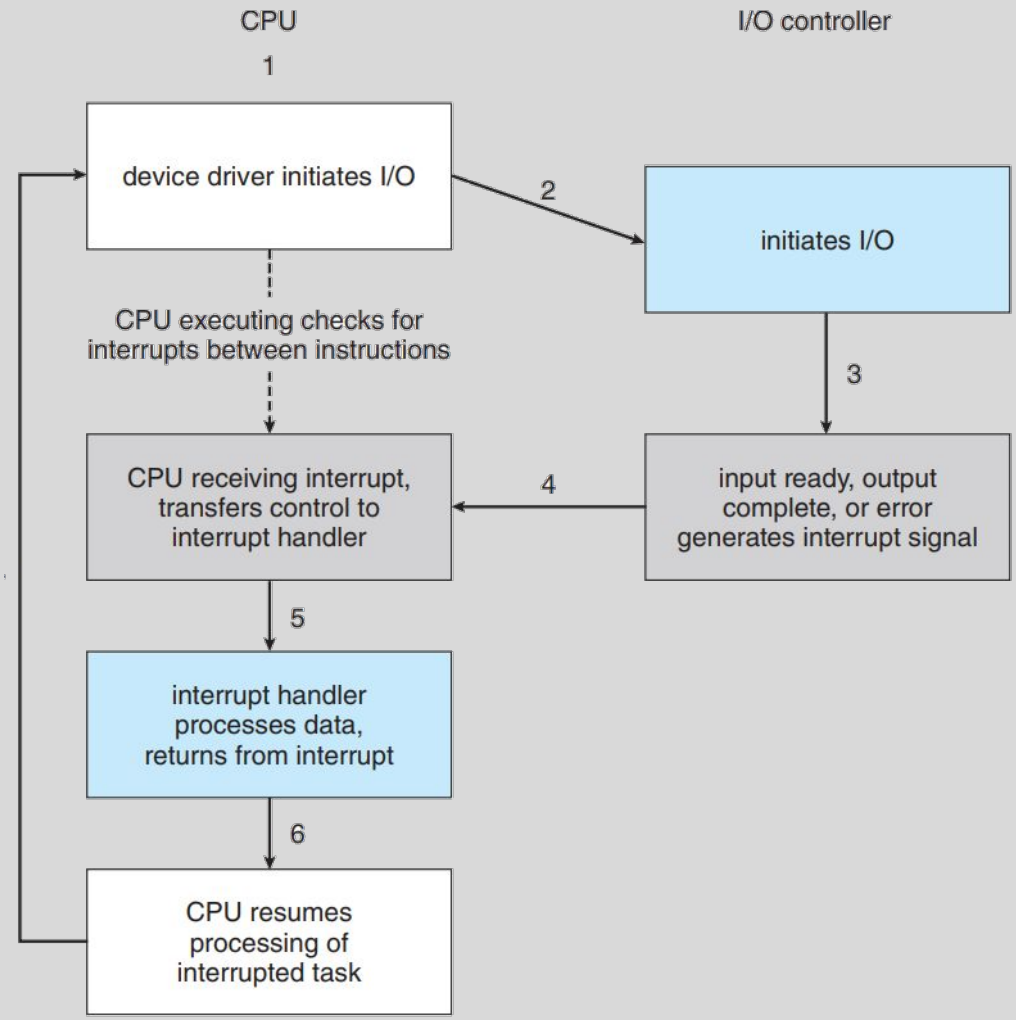
- **Interrupciones**
- **System calls**

Interrupciones

Ejemplo: flujo de una operación de I/O

1. **Inicio**: El *Device Driver* (software) le da instrucciones al *Device Controller* (hardware) cargando datos en sus registros (ej: "necesito leer una tecla").
2. **Acción**: El controlador analiza la orden y espera el evento físico. Cuando el usuario presiona una tecla, el controlador mueve ese dato desde el teclado hacia su propio búfer local (una pequeña memoria interna del controlador).
3. **Aviso**: Una vez que el dato está listo en el búfer, el controlador debe avisar que terminó.
4. La **Interrupción**: El controlador lanza una interrupción hacia la CPU. Es como un "toque en el hombro" que le dice al sistema: "Ya tengo el dato, vení a buscarlo".

En resumen: El Driver pide, el Controlador ejecuta y, finalmente, la Interrupción es la señal que cierra el ciclo avisando que el trabajo está hecho.

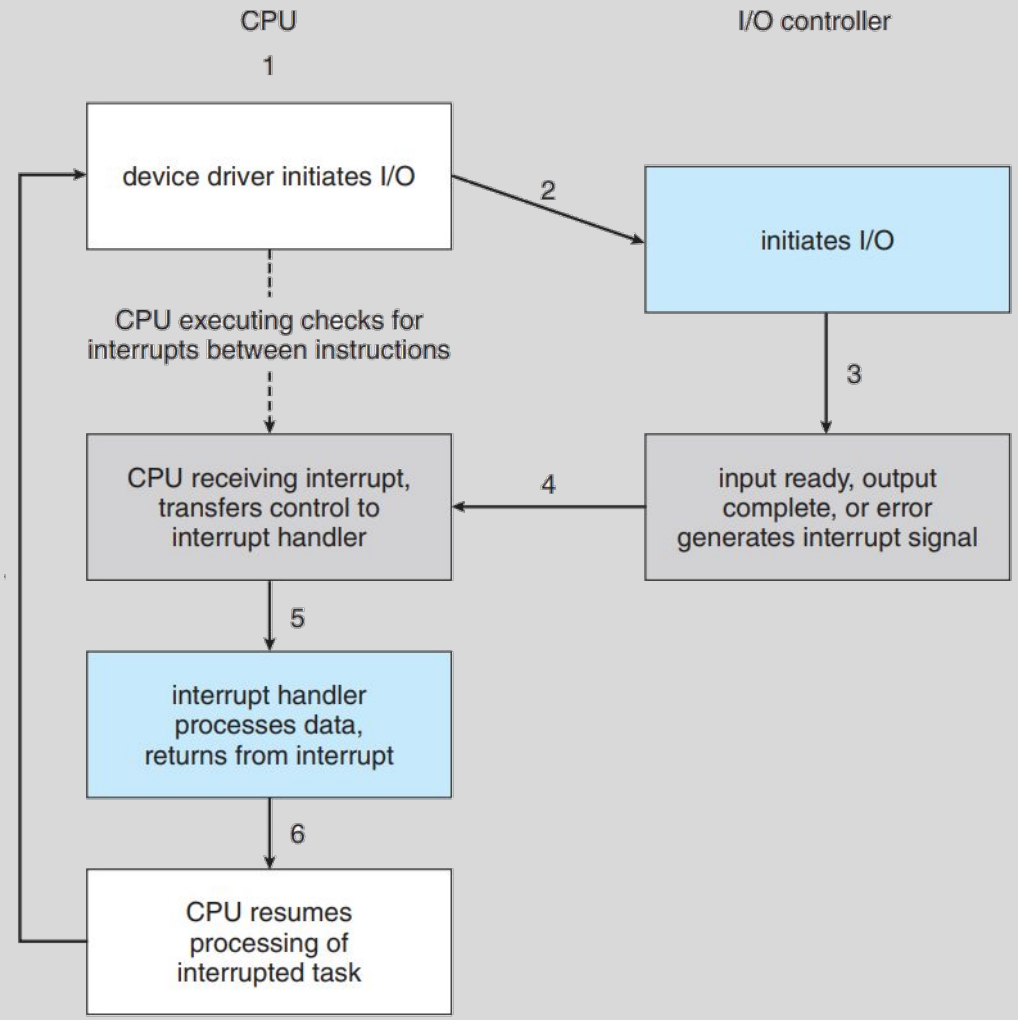


Ciclo de una interrupción

1. El hardware envía una señal eléctrica a la CPU en cualquier momento para avisar que necesita atención.

2. La CPU detiene inmediatamente su tarea actual. Para no perder el contexto, guarda el estado del procesador (registros, posición actual del programa, etc.).

3. La ejecución salta a una dirección de memoria fija donde reside la **Rutina de Servicio de Interrupción** (ISR), que es el código encargado de resolver el evento.



Ciclo de una interrupción

4. Se ejecuta la rutina (por ejemplo, leer una tecla o procesar un paquete de red).

5. Una vez terminada la rutina, la CPU **restaura el estado** que había guardado y reanuda el trabajo original exactamente donde lo dejó.

Interrupciones

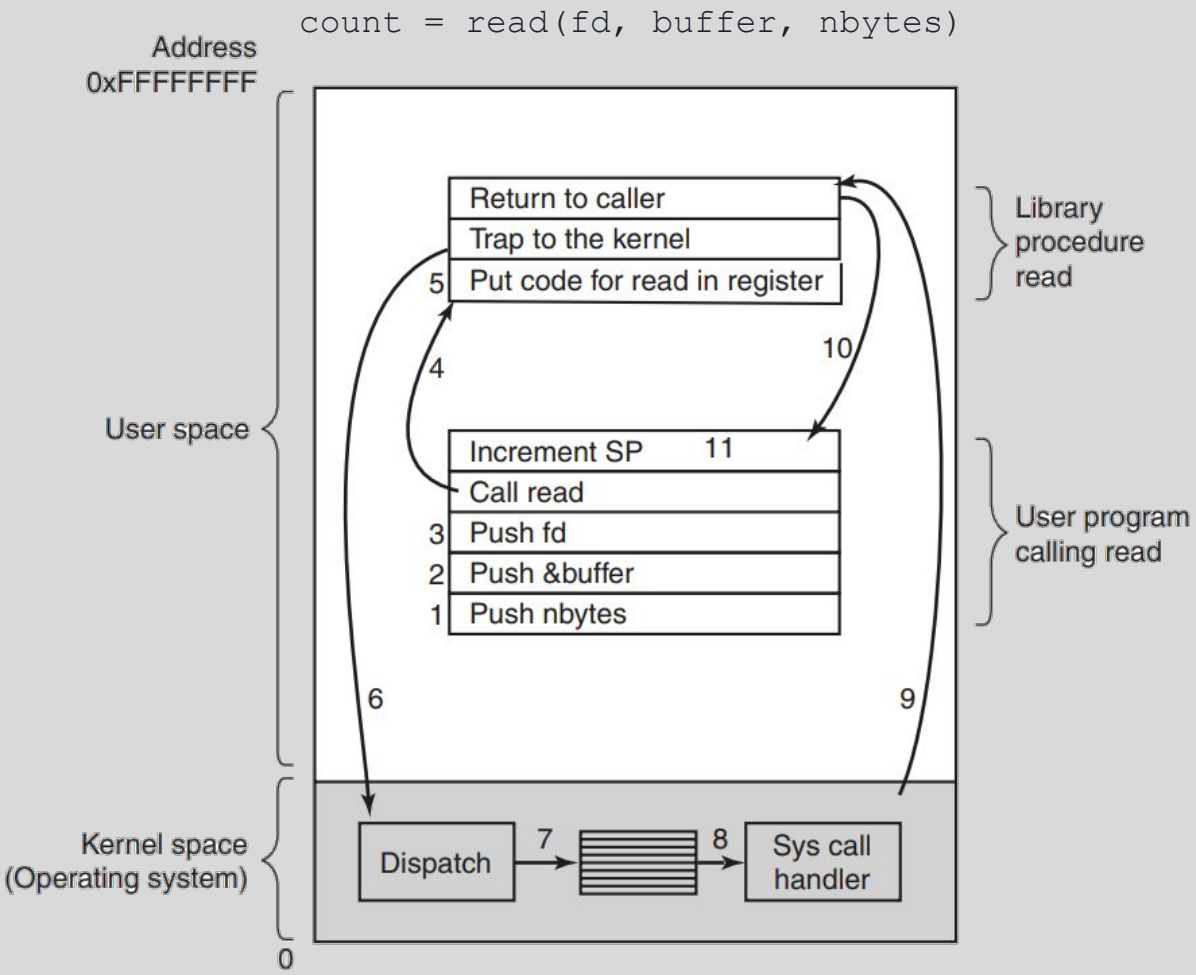
src > 01-introduccion > read_input_example.c > main()

```c
#include <stdio.h>

int main()
{
    char input[100]; // Buffer to store user input

    printf("Enter a value: ");
    if (fgets(input, sizeof(input), stdin) != NULL) {
        printf("You entered: %s", input);
    } else {
        printf("Error reading input.\n");
    }

    return 0;
}
```



System call

El Puente entre el Usuario y el Kernel

1. Cuando un programa necesita hacer algo restringido (como escribir un archivo en el disco), no puede hacerlo solo. Debe pedirle permiso al SO.
2. El programa ejecuta una instrucción especial que provoca un "salto" a una ubicación específica en la tabla de interrupciones.
3. El SO toma el control, verifica que el programa tenga permisos para esa tarea, la realiza en su nombre y luego le devuelve el resultado.
4. Una vez terminada la tarea, el control regresa al programa del usuario para que continúe su ejecución normal.

System call

src > 01-introduccion > read_file_example.c > main()

```c
#include <stdio.h>
#include <stdlib.h>

#define BUFFER_SIZE 1024

int main()
{
  FILE *file;
  char buffer[BUFFER_SIZE];

  // Open file
  file = fopen("../../recursos/tup.txt", "r");
  if (file == NULL) {
    perror("Error opening file");
    return 1;
  }

  printf("Contenido del archivo TUC.txt:\n");
  printf("-------------------------------\n");

  // Read and print file content
  while (fgets(buffer, BUFFER_SIZE, file) != NULL) {
    printf("%s", buffer);
  }

  // Close file
  if (fclose(file) != 0) {
    perror("Error closing file");
    return 1;
  }

  return 0;
}
```

PROCESOS

Bibliografía

- Operating System Concepts. By Abraham, Silberschatz.
  - Capitulo III

Procesos

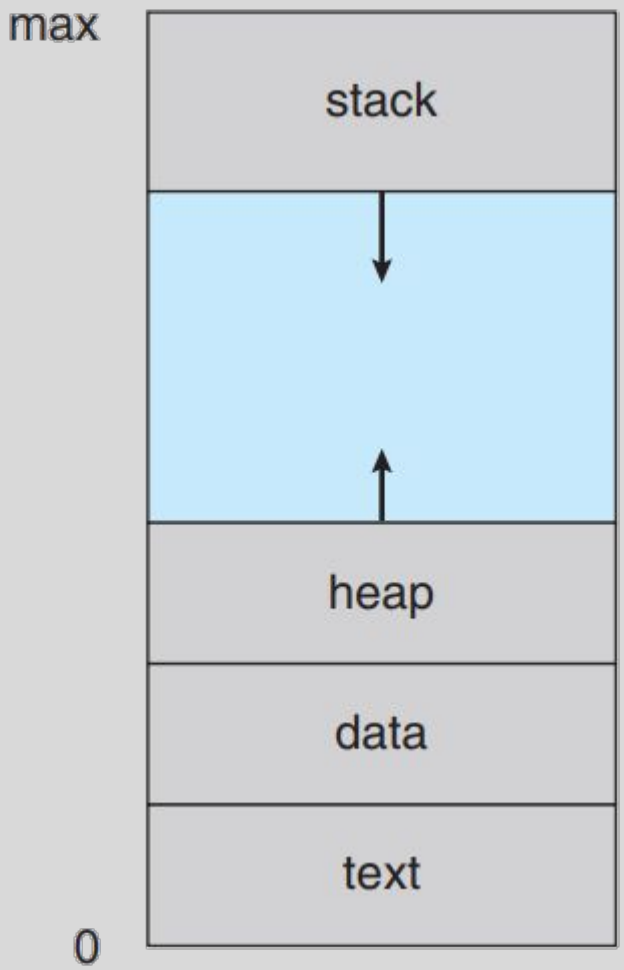
Programa vs. Proceso

1. **Programa** (entidad **Pasiva**): Es simplemente un archivo guardado en el disco. Es una lista de instrucciones que no hace nada hasta que alguien la invoca.
2. **Proceso** (entidad **Activa**): Es el programa cuando ya se está ejecutando. Es la **unidad de trabajo** real del Sistema Operativo.
3. Componentes del Proceso: Para funcionar, un proceso no solo tiene el código, sino que incluye:
    a. PC (Program Counter): Un puntero que indica cuál es la próxima instrucción que debe ejecutarse.
    b. Recursos: El espacio en memoria, los registros del procesador y los archivos que tiene abiertos.

Convivencia en el Sistema

Un sistema operativo moderno maneja una colección de muchos procesos a la vez:

- Procesos de **Usuario**: Los que un usuario normal ejecuta (como el navegador).
- Procesos del **Sistema**: Los que el propio SO necesita para funcionar.

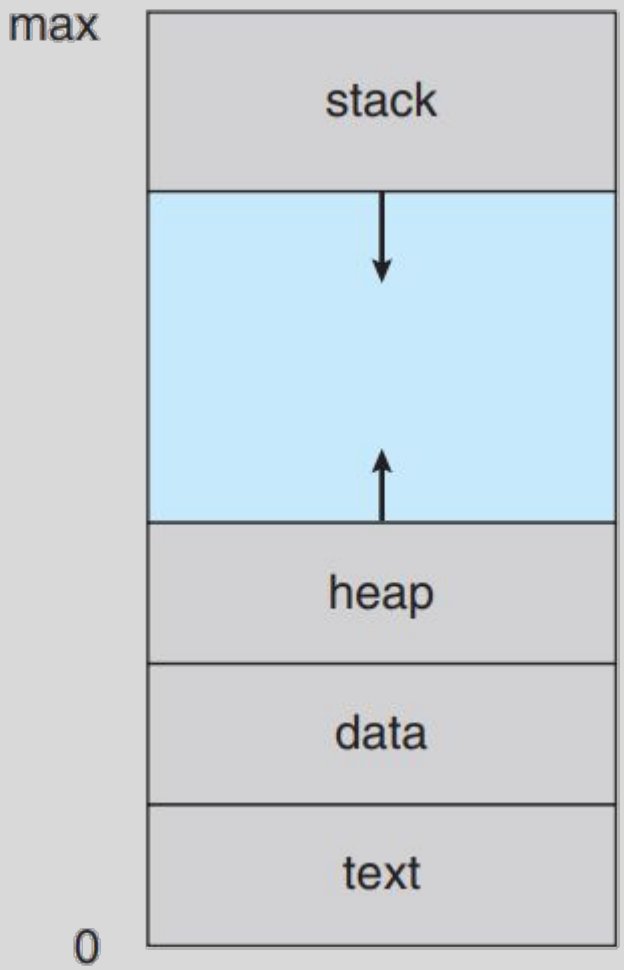


Representación de un Proceso en Memoria

Para que un programa funcione, el SO divide su espacio de memoria en cuatro secciones principales:

Secciones Estáticas (Tamaño Fijo):

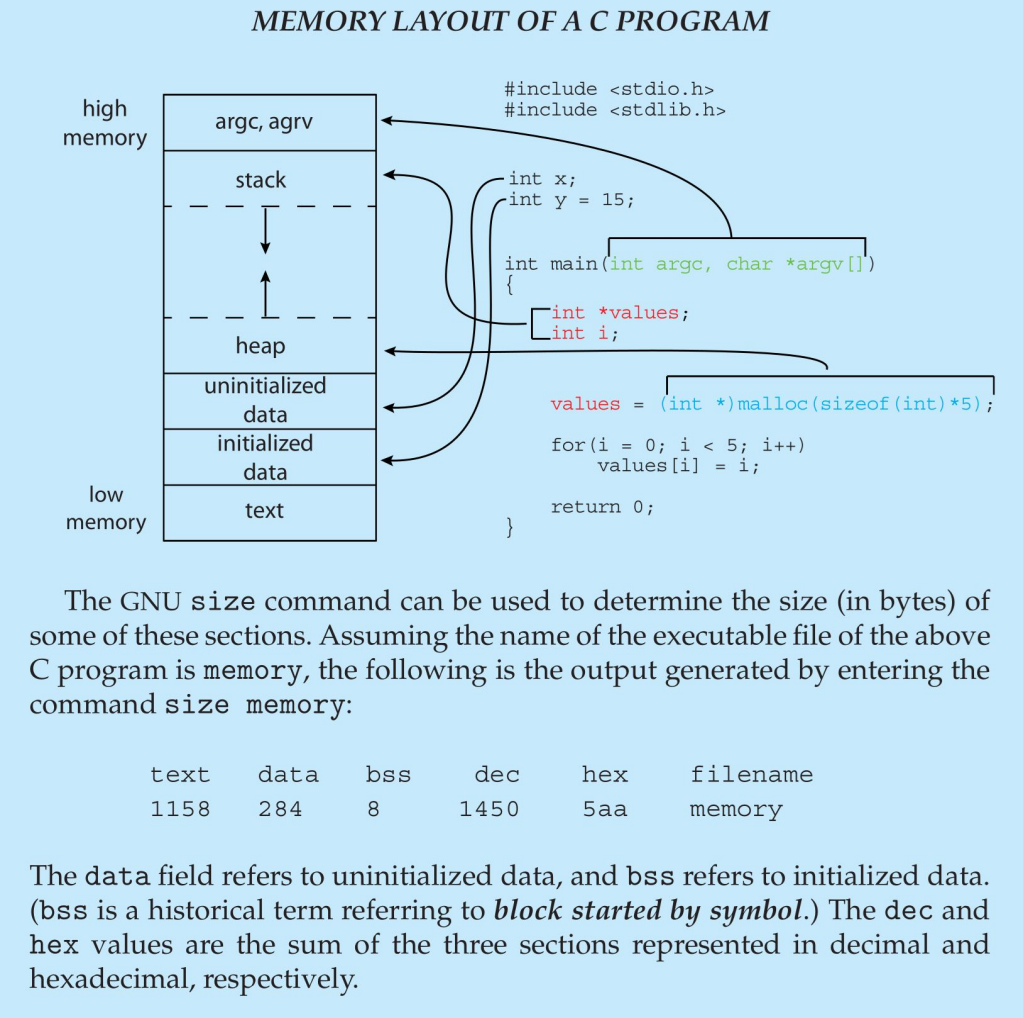
- **Text**: Contiene el código ejecutable (las instrucciones reales que lee la CPU).
- **Data**: Guarda las variables globales y estáticas definidas antes de arrancar.



Representación de un Proceso en Memoria

Secciones Dinámicas (Crecen y se Achican):

- **Heap**: Memoria que el programa solicita "a demanda" mientras corre (por ejemplo, al crear objetos nuevos).
- **Stack**: Almacenamiento temporal para cuando se llaman funciones. Ahí se guardan parámetros, variables locales y la dirección para saber a dónde "volver" cuando la función termina.

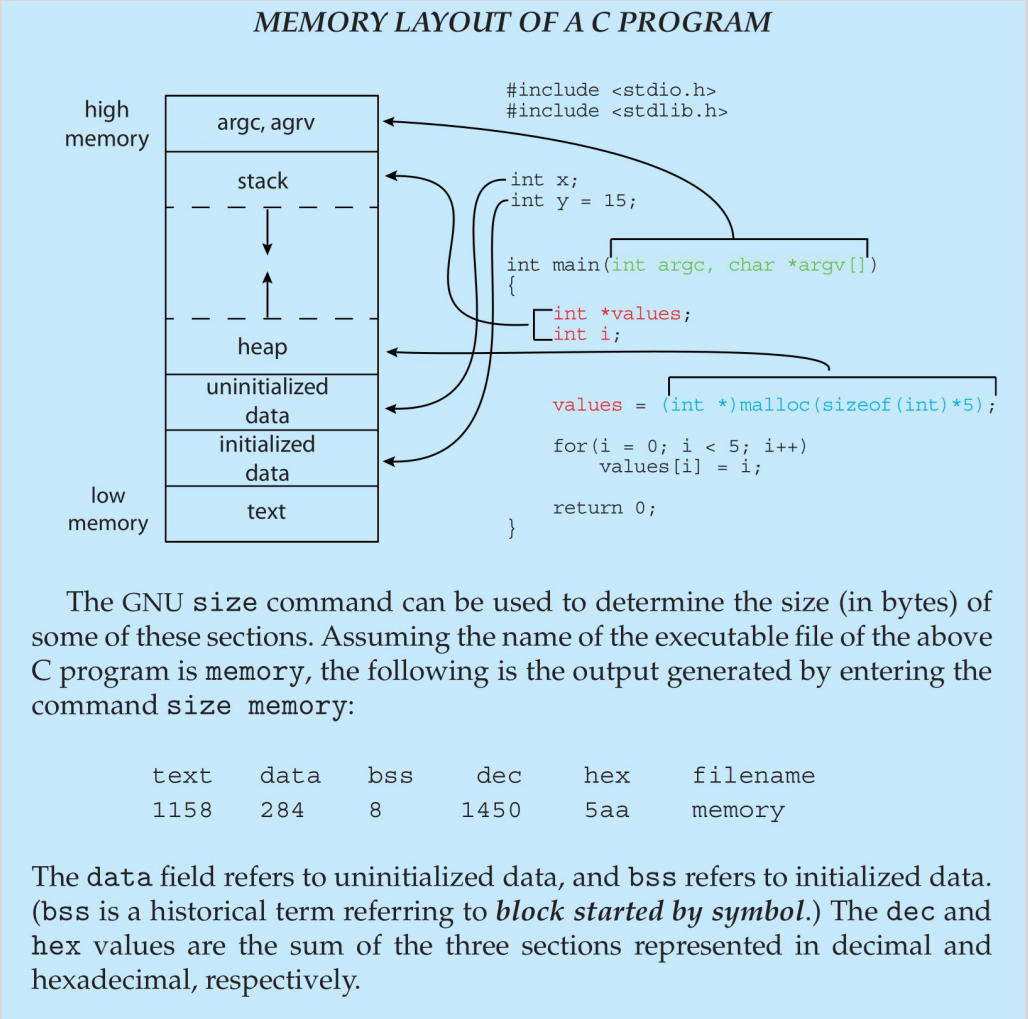


Organización Detallada de un Proceso (Ejemplo en C)

Esquema real de un programa en ejecución:

1. Sección de Argumentos (*argc*, *argv*): Situada en la parte más alta de la memoria (*high memory*). Almacena los parámetros que pasan al programa cuando se ejecuta.
2. *Stack* y *Heap* (Dinámicos): *Stack* crece hacia abajo al llamar funciones. *Heap* crece hacia arriba cuando el programador necesita “almacenar” valores del programa.

Organización Detallada de un Proceso (Ejemplo en C)



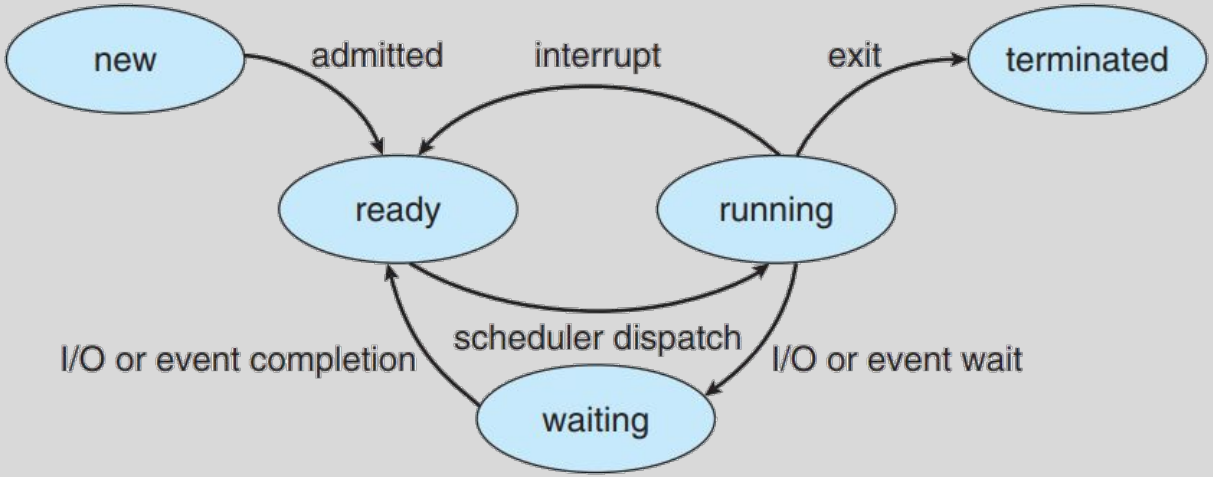
3. División de Datos (*Data*): Se separa en dos bloques para ser más eficiente:
   - *Initialized Data*: Para variables globales con un valor asignado (ej: `int y = 15;`).
   - *Uninitialized Data* (BSS): Para variables globales que no tienen valor inicial (ej: `int x;`).
4. Sección *Text*: La base del proceso, donde reside el código de la aplicación.

Estados

Mientras se ejecuta, el proceso cambia de estado.

- **Nuevo**: el proceso es creado.
- **Ejecutando**: las instrucciones están siendo ejecutadas.
- **Esperando**: el proceso espera la ocurrencia de un evento.
- **Listo**: el proceso espera ser asignado a un procesador.
- **Terminado**: el proceso ha finalizado su ejecución.

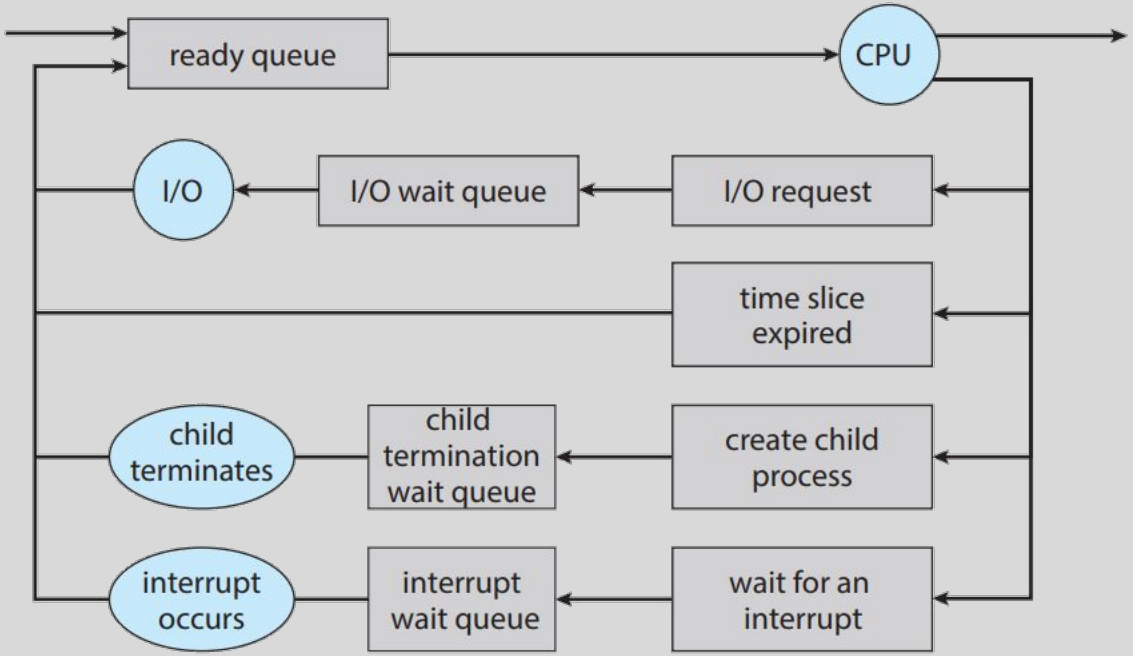
Es importante notar que **solo un proceso** puede estar ejecutándose en un núcleo del procesador en un instante. Sin embargo, muchos procesos pueden estar listos y esperando.



Ciclo de Vida del Proceso

Todo proceso comienza en la **cola de listos** (***Ready Queue***) esperando su turno. Una vez que el planificador le asigna un núcleo de CPU y comienza a ejecutarse, su camino puede tomar cuatro direcciones:

1. **Espera de E/S (*I/O*)**: Si el proceso necesita un dato (del disco, teclado, etc.), abandona la CPU y se mueve a una **cola de espera de E/S** hasta que el dispositivo responda.

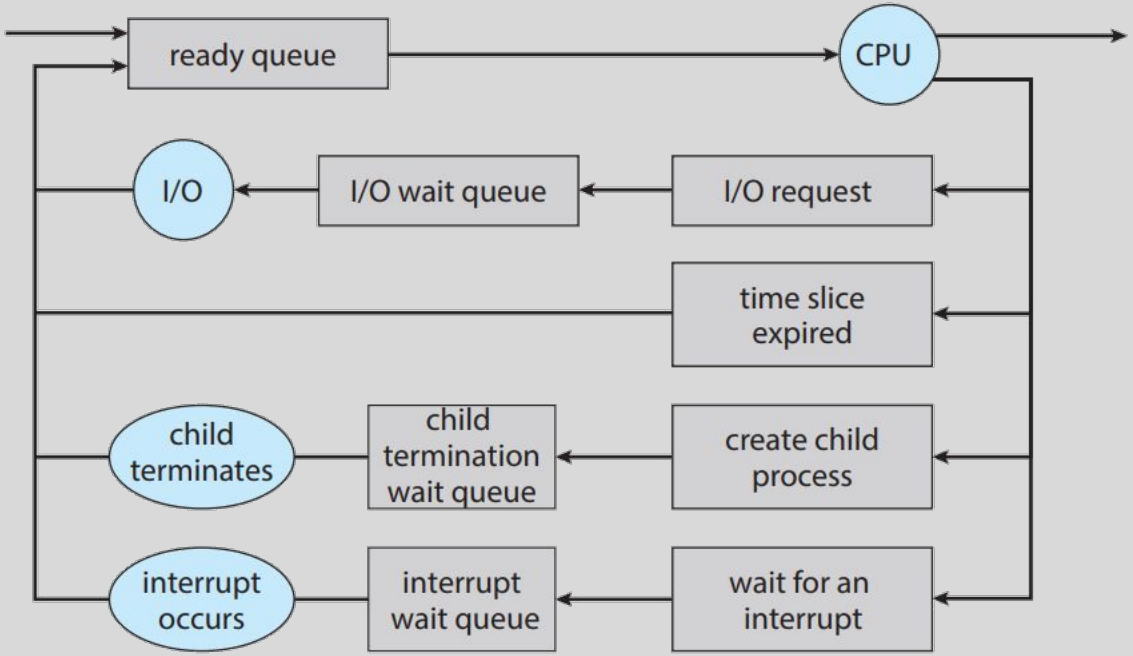


Ciclo de Vida del Proceso

2. **Fork**: Si el proceso lanza un nuevo proceso (hijo), suele pausarse y entrar en una cola de espera hasta que el hijo termine su tarea.

3. **Interrupción o Tiempo Agotado**: El SO puede quitarle la CPU a la fuerza si se acaba su tiempo asignado (*quantum*) o si llega una interrupción de mayor prioridad.

4. **Finalización**: Si el proceso completa todas sus instrucciones, se marca como terminado. El SO limpia la memoria, elimina su PCB y libera todos los recursos que estaba usando.



Bloque de Control (PCB)

Cada proceso es representado en el SO por un **Bloque de Control de Procesos** (*Process Control Block*). El cual contiene mucha información asociada a un proceso:

- Estado del Proceso (new, ready, etc.)
- PC
- Registros del CPU.
- Información sobre la Planificación del CPU.
- Información sobre la Administración de Memoria.
- Información contable (cantidad de CPU, pid, etc).
- Información del Estado E/S.

ADMINISTRACIÓN Y PLANIFICACIÓN DE PROCESOS

Planificación de Procesos

Multiprogramación vs. Tiempo Compartido (*Timesharing*)

- **Multiprogramación**: Su objetivo es la **eficiencia**. Busca que la CPU nunca esté ociosa. Si un proceso se detiene (por ejemplo, esperando un dato del disco), el SO le entrega la CPU a otro inmediatamente.
  - Dato: El número de procesos en la RAM se llama **grado de multiprogramación**.
- **Timesharing**: Su meta es la **interactividad**. Cambia de un proceso a otro tan rápido que el usuario siente que todos sus programas corren al mismo tiempo en su propia PC.

El Planificador de Procesos (*Scheduler*)

- Es el "director de orquesta" que elige qué proceso de la lista de los que están listos (***ready***) pasa a ocupar un núcleo de la CPU en cada momento.

Planificación de Procesos

Tipos de Procesos según su comportamiento

- Los procesos suelen dividirse en dos categorías según qué recurso usan más:
  - **Limitados por E/S** (***I/O-bound***): Pasan la mayor parte del tiempo esperando una entrada o salida (teclado, disco, red). Hacen cálculos muy breves.
  - **Limitados por CPU** (*CPU-bound*): Pasan la mayor parte del tiempo haciendo cálculos matemáticos o lógicos complejos y piden muy poca intervención de periféricos.

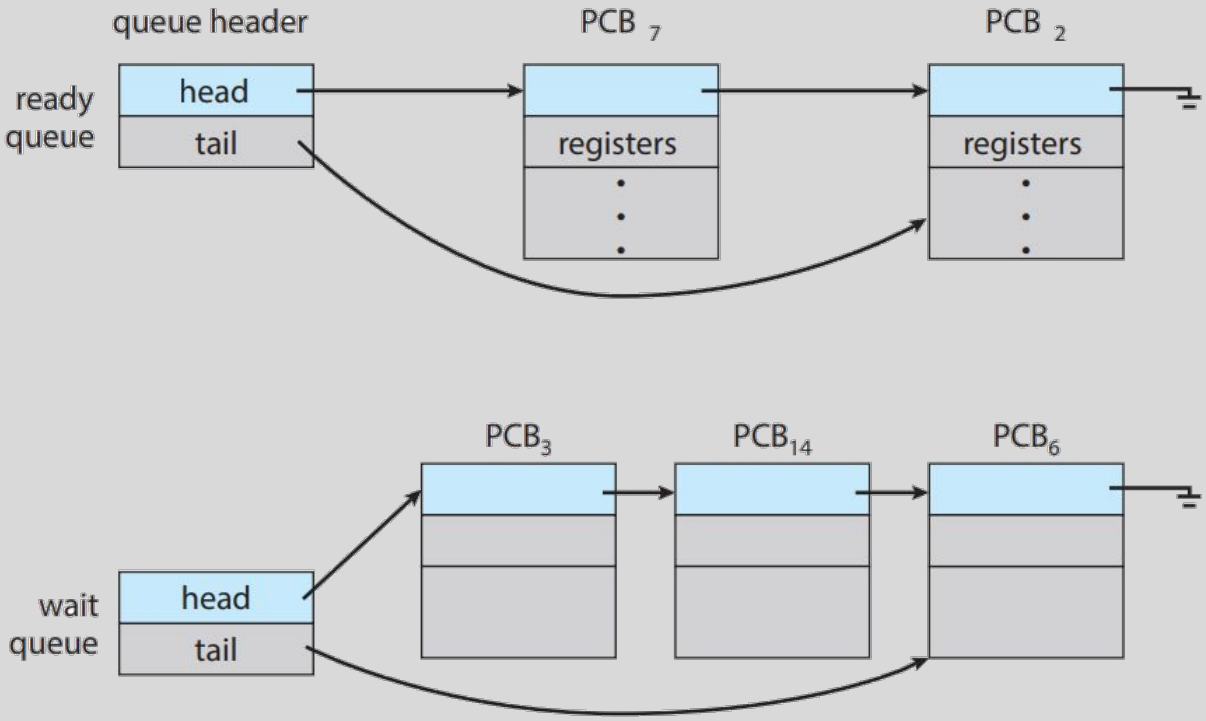
En resumen: El SO combina estas técnicas para que la computadora sea rápida (**multiprogramación**) y responda al instante a tus clics (**timesharing**), sin importar si el programa es pesado para el procesador o para el disco.

Colas de Planificación

Cuando un proceso está cargado en memoria y solo necesita de la CPU para empezar a trabajar, se coloca en la **cola de listos**.

Para gestionar esta espera de forma eficiente, el SO utiliza una **lista enlazada**, donde mantiene un **Header**: es un puntero que guarda la dirección del primer PCB del proceso en la fila. Y un **Tail**: un puntero al último PCB.

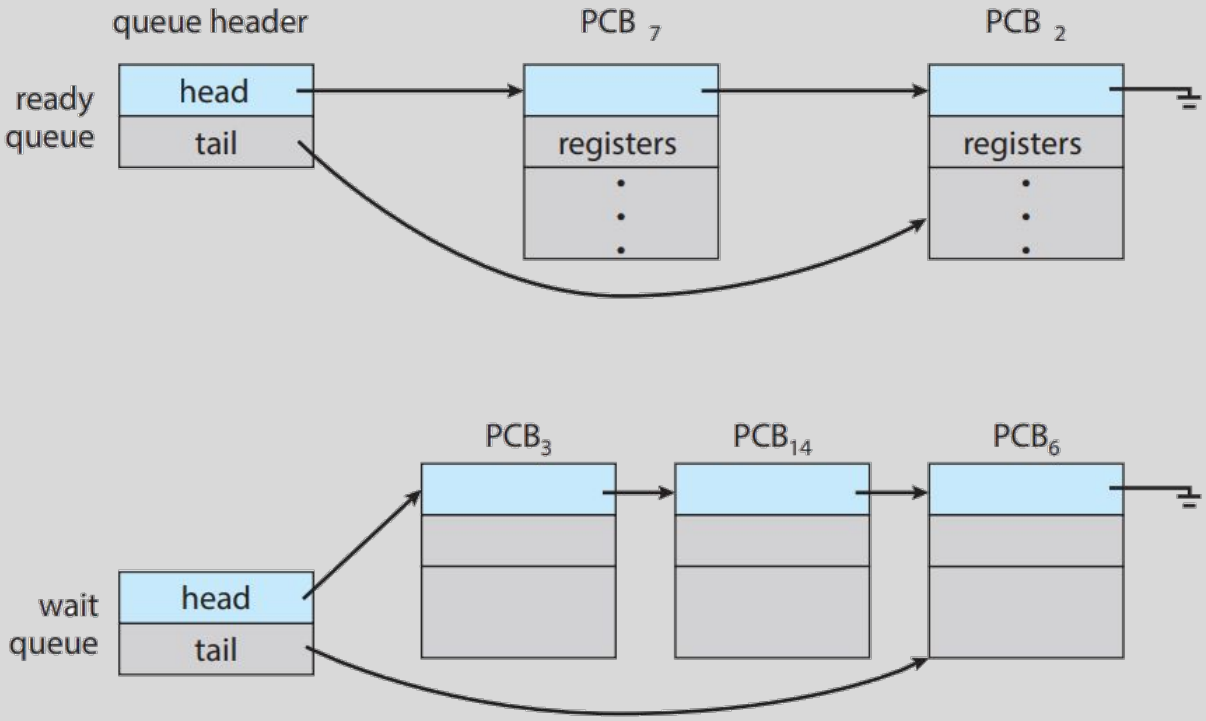
Dentro de cada PCB hay un campo especial que apunta a la siguiente PCB en la lista. El último proceso de la fila apunta a "vacío" (*null*), indicando que ahí termina la cola.



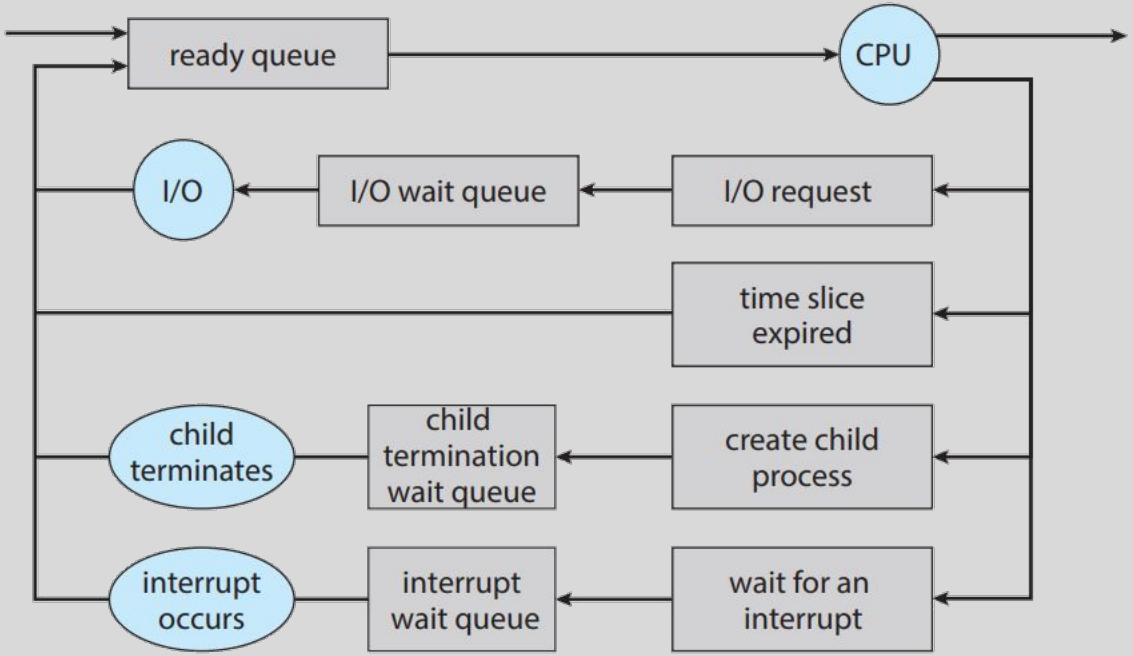
Colas de Planificación

No todos los procesos en el sistema están listos para correr.

1. Cuando un proceso solicita una operación de E/S, el planificador lo saca de la CPU y lo mueve a una **cola de espera** específica para ese dispositivo.
2. El proceso permanece ahí, "dormido", hasta que ocurre el evento que necesita (por ejemplo, que el disco termine de leer un archivo).
3. Una vez que el dispositivo avisa que terminó (mediante una **interrupción**), el proceso sale de la cola de espera y vuelve a la **cola de listos** para competir otra vez por un turno en la CPU.



Ciclo de Vida del Proceso



Planificador y el Cambio de Contexto

El **Planificador de la CPU** (*CPU Scheduler*) es el motor que mantiene el sistema en movimiento, tomando decisiones críticas a una velocidad extremadamente rápida (generalmente cada pocos milisegundos).

1. **Estrategia de Selección**

El planificador no trata a todos los procesos por igual, sino que se adapta a su naturaleza:

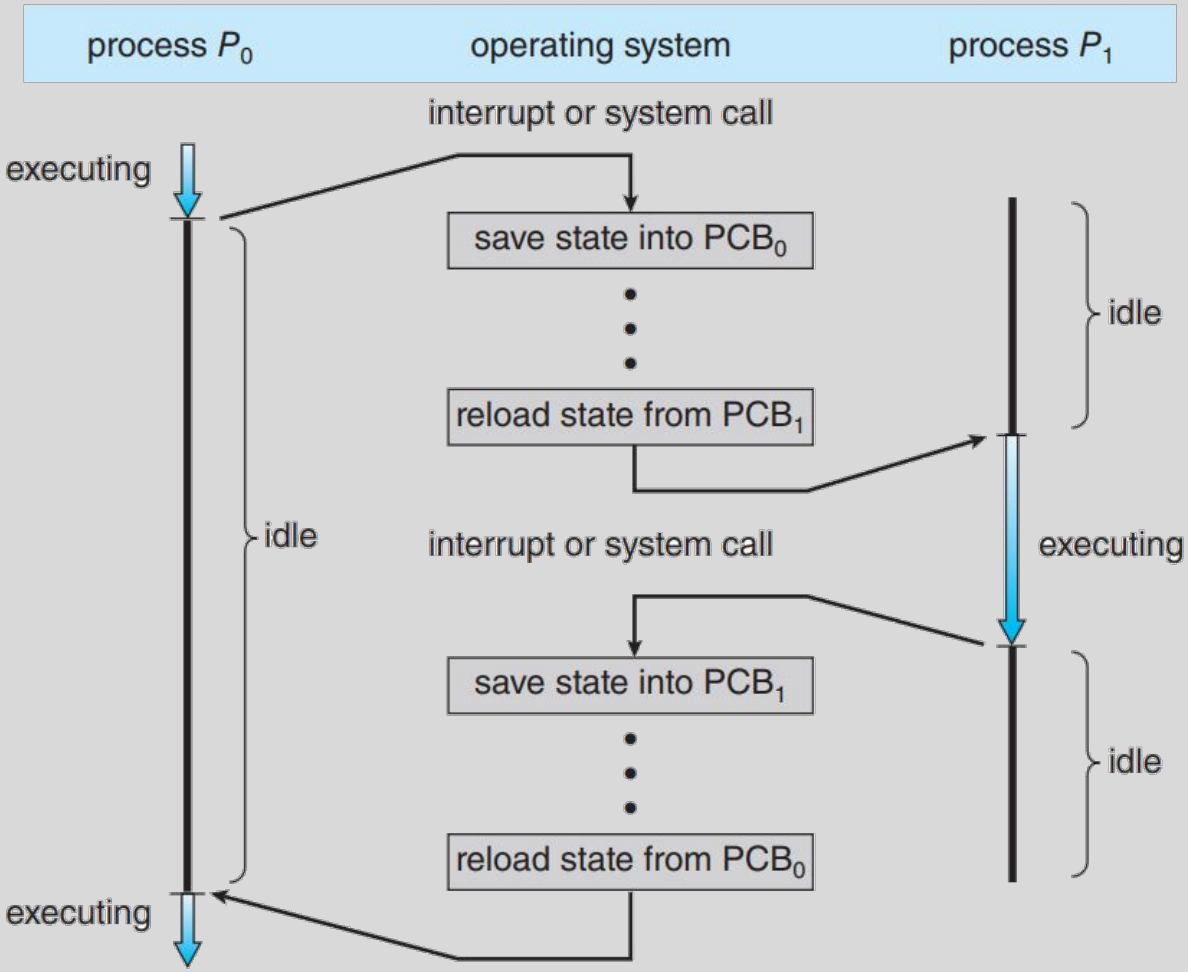
- Procesos de E/S (I/O-bound): Como suelen pedir datos rápido, el planificador los deja entrar, pero estos abandonan la CPU en apenas unos milisegundos para ir a una cola de espera.
- Procesos de CPU (CPU-bound): Aunque quieran quedarse mucho tiempo calculando, el planificador les pone un límite. Si se pasan de su tiempo asignado, los expulsa para que otros tengan oportunidad.

Planificador y el Cambio de Contexto

2. **Frecuencia de Ejecución**
Para que el usuario sienta que todo fluye, el planificador actúa muchas veces por segundo (mínimo una vez cada 100 ms, pero suele ser mucho más rápido, aproximadamente cada 10 ms). Esta alta frecuencia es lo que permite que la computadora sea interactiva.

3. **Cambio de Contexto** (*Context Switch*)
Cada vez que el planificador decide quitarle la CPU a un proceso y dársela a otro, ocurre un **Cambio de Contexto**:

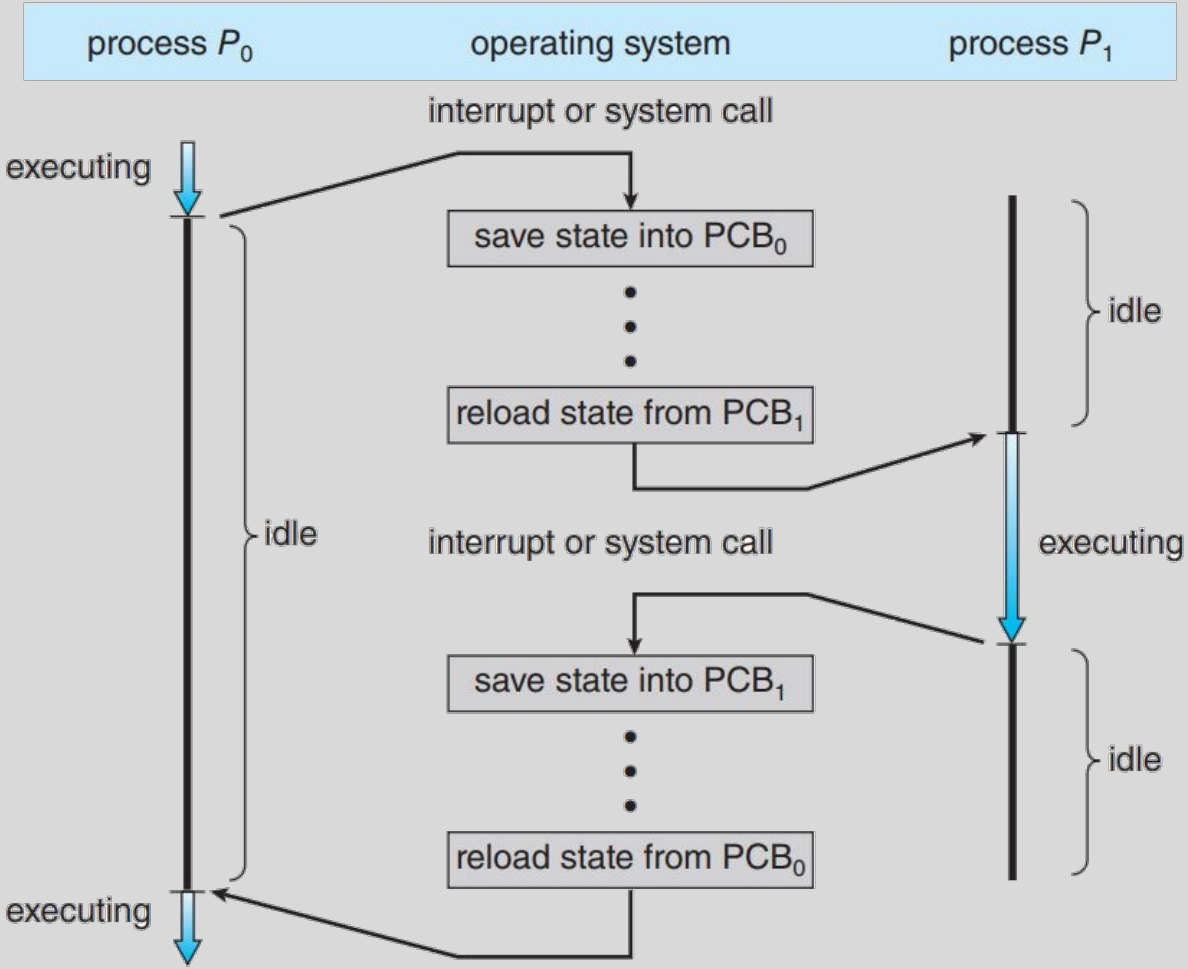
1. **Guardar**: El SO salva el "estado" actual del proceso que sale (su PC, registros y memoria) en su PCB.
2. **Cargar**: El SO busca el PCB del nuevo proceso y carga su último estado guardado en los registros de la CPU.
3. **Ejecutar**: El nuevo proceso continúa exactamente donde se había quedado.



Cambio de Contexto

Es el proceso técnico que permite al SO suspender un proceso y reanudar otro (o el mismo más tarde) sin que se pierda el progreso.

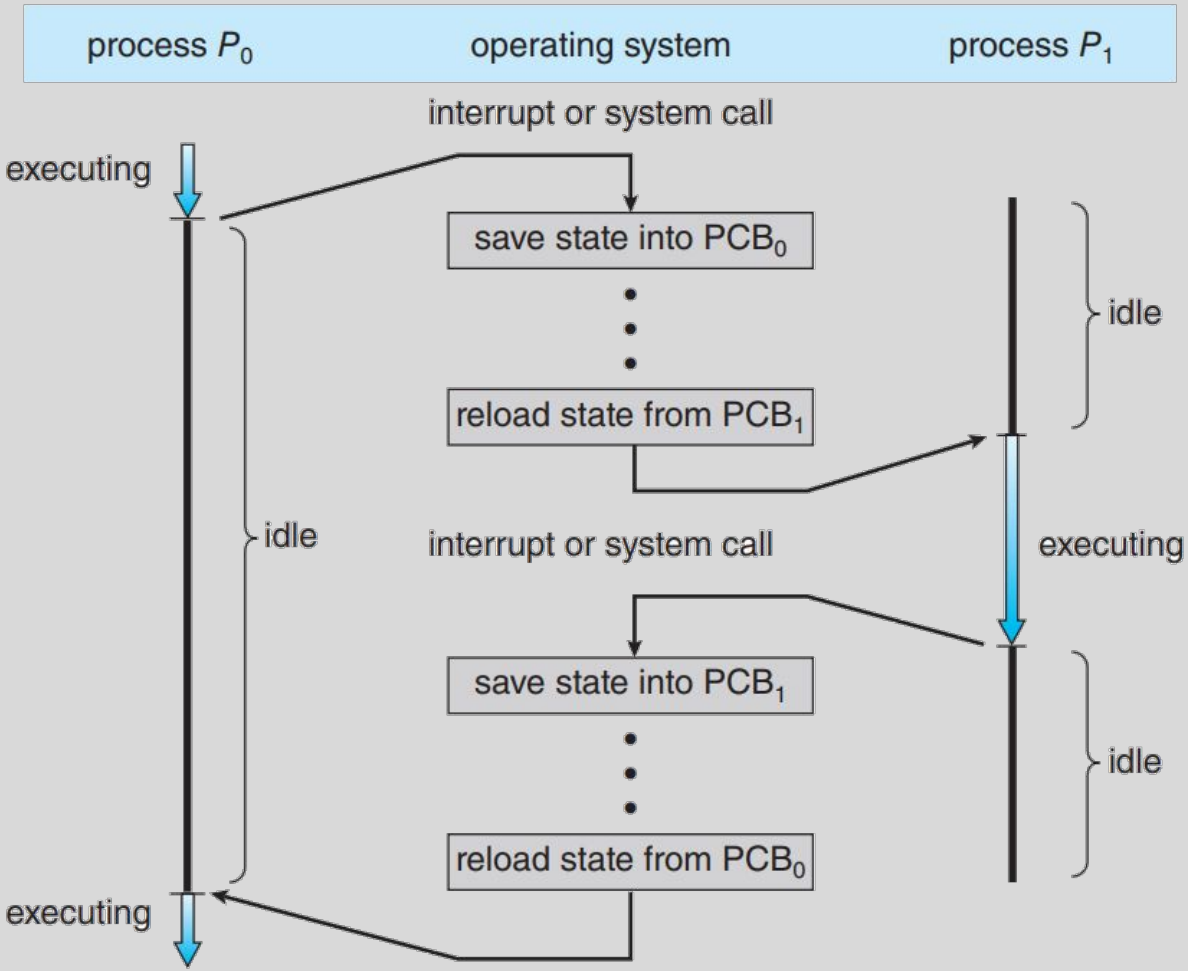
1. Cuando llega una interrupción, el sistema debe "congelar" el estado actual del núcleo de la CPU.
   - Se guardan los valores de los registros, el PC y el modo de ejecución (kernel o usuario).
   - Toda esta información se almacena en el PCB del proceso que se está interrumpiendo.



Cambio de Contexto

2. El kernel realiza una maniobra de dos pasos:

a. Guarda el contexto del proceso saliente en su PCB.
b. Busca el PCB del nuevo proceso programado y carga sus valores guardados en los registros de la CPU.



Cambio de Contexto

3. El cambio de contexto es **tiempo perdido** (*overhead* puro):

- Mientras el kernel cambia los contextos, la CPU no está ejecutando ninguna instrucción del usuario ni resolviendo cálculos; solo está haciendo "trabajo" administrativo.
- Suele durar unos pocos **microsegundos**, pero ocurre tantas veces por segundo que el impacto total es significativo.
- La velocidad final depende de qué tan eficiente sea la arquitectura de la computadora para mover estos datos rápidamente.

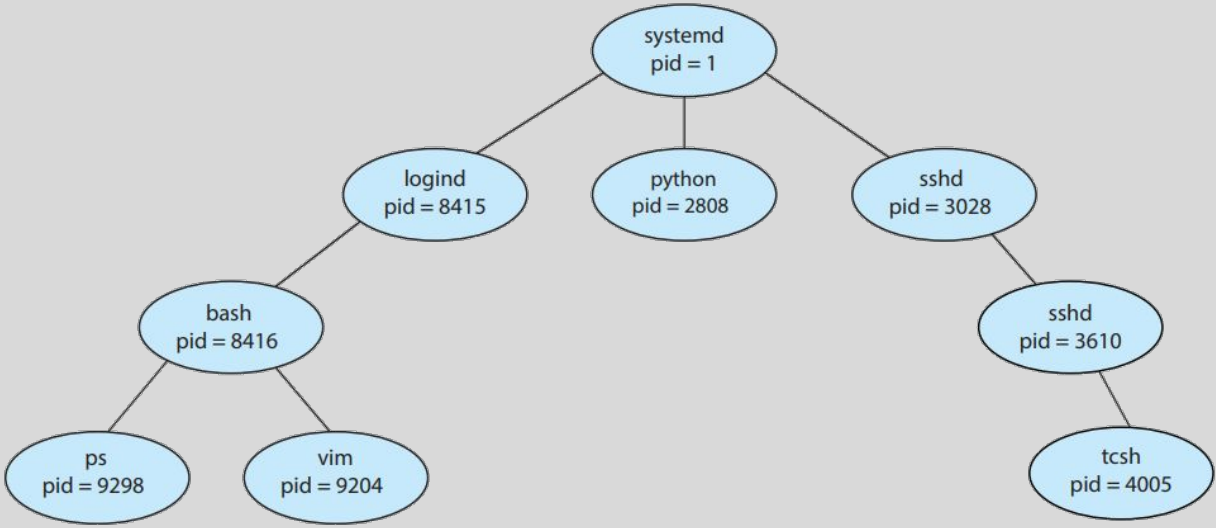
OPERACIONES SOBRE PROCESOS

Creación de Procesos

La Jerarquía: Padres e Hijos

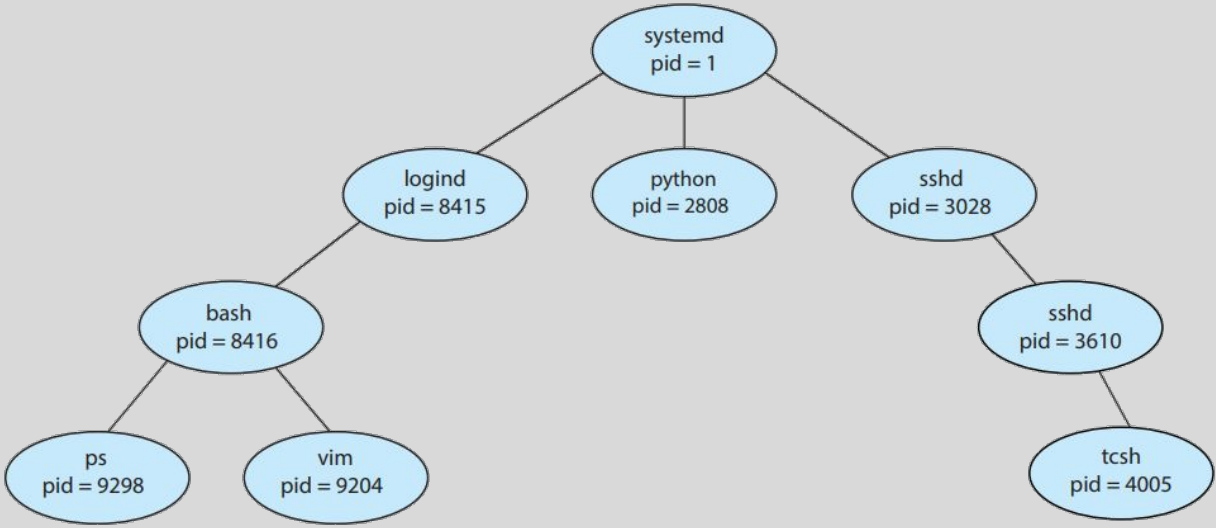
En el sistema operativo, los procesos no nacen aislados, sino que forman un **árbol de procesos**:

- **Proceso Padre**: El creador del nuevo proceso.
- **Proceso Hijo**: El nuevo proceso creado, que a su vez puede ser padre de otros.
- **PID** (*Process Identifier*): Un número entero único que el kernel asigna a cada proceso para identificarlo y acceder a sus atributos.



Creación de Procesos

- El Ancestro Común (`systemd`): En sistemas como Linux, el primer proceso que se crea al arrancar es `systemd` (siempre con **PID = 1**). De este derivan todos los demás servicios (servidores web, SSH, impresión, etc.).

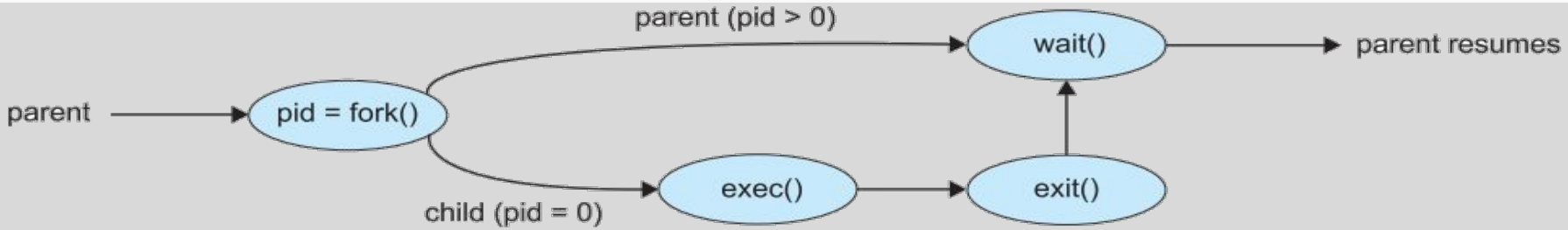


Creación de Procesos

Mecanismos de Creación: `fork()` y `exec()`

El nacimiento de un proceso ocurre generalmente en dos etapas técnicas:

- `fork()` (**Clonación**): Crea un proceso hijo que es un **duplicado exacto** del padre (misma memoria, mismos datos).
  - Ambos siguen ejecutando la instrucción justo después del `fork()`.
  - **Diferenciación**: Para saber quién es quién, `fork()` devuelve **0** al hijo y el **PID del hijo** al padre.
- `exec()` (**Transformación**): Normalmente, ni bien termina la clonación, el hijo llama a una función de la familia `exec()`. Esta función destruye la imagen de memoria copiada del padre y carga un **nuevo programa binario**. El proceso "se convierte" en algo distinto.



Creación de Procesos

Gestión de Recursos y Ejecución

Cuando se crea un nuevo proceso, necesita recursos para realizar su tarea (CPU, memoria, archivos):

- **Recursos**: Puede pedirlos directamente al SO o estar limitado a usar una parte de los recursos que ya tiene el padre.
- **Modos de Ejecución**:
  - **Simultánea**: Padre e hijo corren al mismo tiempo.
  - **Espera**: El padre ejecuta la *syscall* `wait()`, generando su salida de la **cola de listos** hasta que el hijo termine su tarea.

Terminación de Procesos

Terminación Normal y Recursos

Un proceso termina voluntariamente cuando ejecuta su última instrucción o llama a la *syscall* `exit()`. En este momento, el Sistema Operativo realiza una limpieza:

- Libera la **memoria** (física y virtual).
- Cierra los **archivos** que el proceso tenía abiertos.
- Limpia sus **búfers de E/S**.
- El proceso puede devolver un valor de estado (éxito o error) a su padre.

Terminación de Procesos

Terminación Forzada (*Abort*)

Un padre puede decidir “terminar” a un hijo por varias razones:

- **Abuso de recursos**: El hijo está consumiendo más de lo permitido.
- **Inutilidad**: La tarea que el hijo hacía ya no es necesaria.
- **Terminación en Cascada**: Algunos SO no permiten que existan hijos "huérfanos". Si el padre muere, el sistema inicia una reacción en cadena eliminando a todos sus descendientes.

Terminación de Procesos

Estados Especiales: Zombis y Huérfanos

Cuando un proceso muere, no desaparece instantáneamente de la lista del kernel. Aquí aparecen dos conceptos clave:

- **Proceso Zombi**: Es un proceso que ya terminó (`exit`), pero su padre aún no ha preguntado por su estado (no ha llamado a `wait`). El proceso está "muerto", pero su entrada en la tabla de procesos sigue ahí ocupando espacio. Notar que casi todos los procesos son zombis por una fracción de segundo.
- **Proceso Huérfano**: Ocurre cuando el padre termina sin esperar a sus hijos. En sistemas como Linux, estos procesos son "adoptados" por el proceso raíz (`systemd` o `init`), quien se encarga de hacer el `wait()` por ellos cuando terminen.

COMUNICACIÓN ENTRE PROCESOS (IPC)

Comunicación entre Procesos

Tipos de Procesos

- **Independientes**: Funcionan aislados, sin compartir datos ni verse afectados por otros.
- **Cooperativos**: Pueden afectar o ser afectados por otros procesos. Trabajan en conjunto para lograr un objetivo.

¿Por qué fomentar la Cooperación?

Existen tres razones principales para que los procesos trabajen juntos:

- **Intercambio de información**: Varios usuarios o procesos necesitan acceder al mismo dato simultáneamente.
- **Aceleración computacional**: En sistemas *multicore*, una tarea grande se divide en subtareas que corren en paralelo para terminar más rápido.
- **Modularidad**: Permite diseñar el sistema como un conjunto de piezas separadas (procesos o hilos) con funciones específicas.

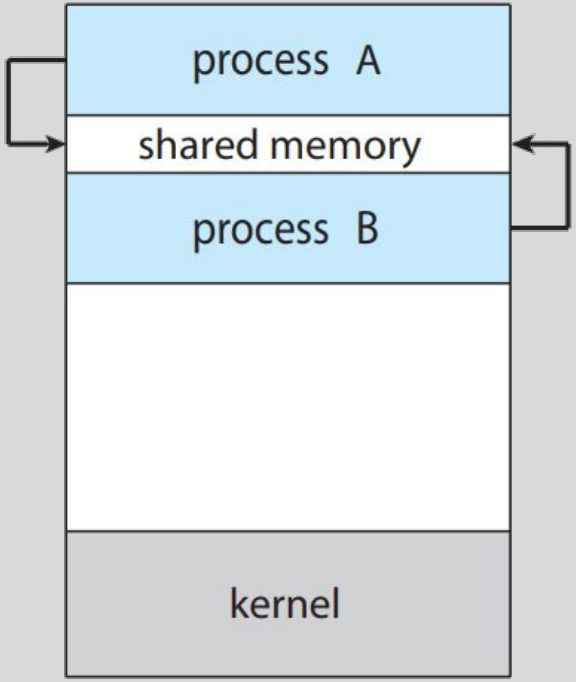
Comunicación entre Procesos

Mecanismos de Comunicación (IPC)

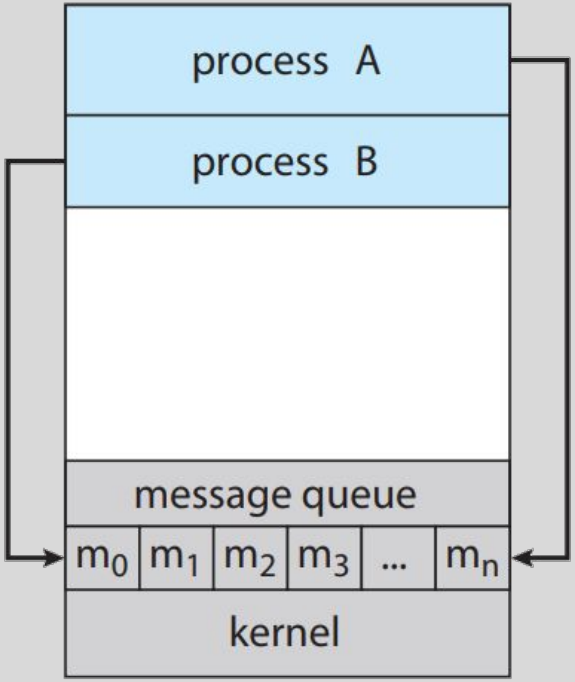
Para cooperar, los procesos usan una técnica llamada *Inter-Process Communication* (IPC). Y existen dos modelos fundamentales:

1. **Memoria Compartida** (*Shared Memory*)

Los procesos se ponen de acuerdo sobre una región de la RAM a la que ambos pueden entrar.



Shared memory.

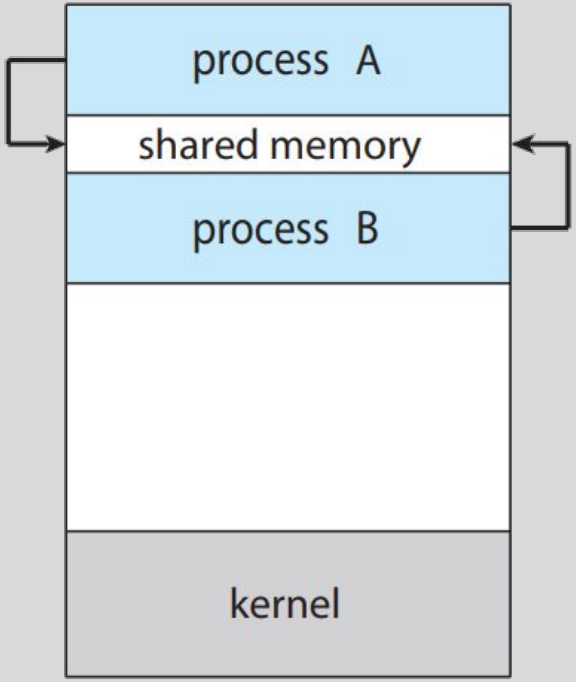


Message passing.

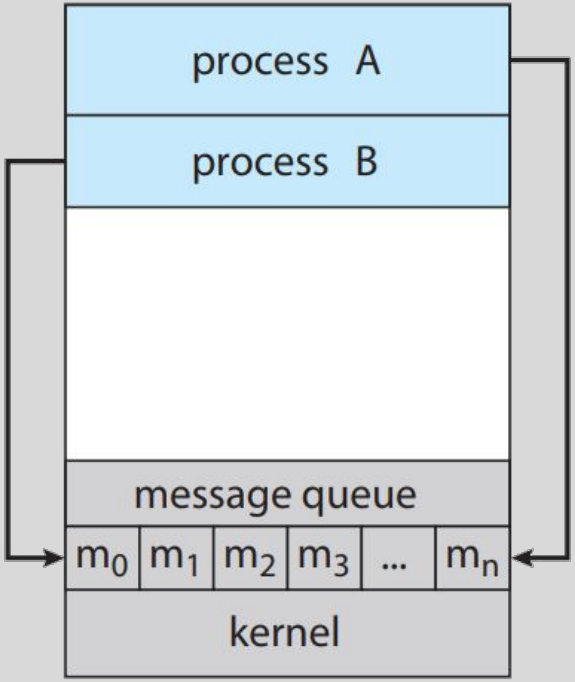
Comunicación entre Procesos

- Uno escribe en esa zona y el otro lee directamente de ella.

- **Ventaja**: Esto es **muy rápido**. Solo se necesita la intervención del kernel (vía una *syscall*) para crear la región. Luego, los procesos acceden a ella como si fuera su propia memoria, sin la intervención del SO.



Shared memory.



Message passing.

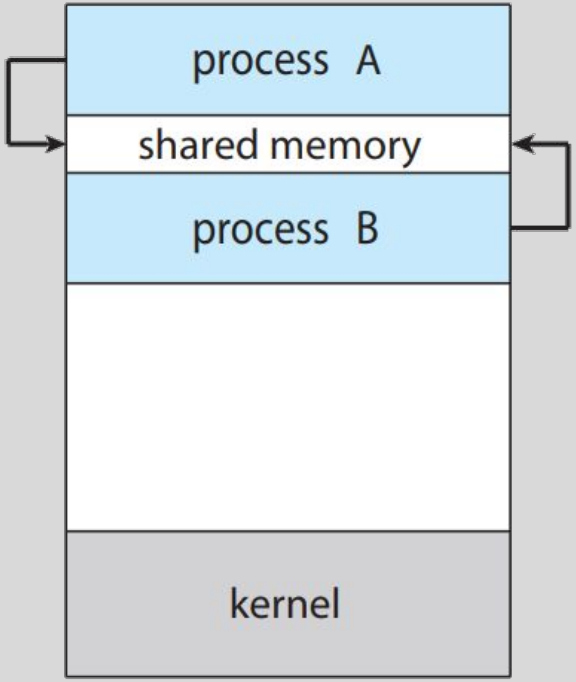
Comunicación entre Procesos

2. **Paso de Mensajes (*Message Passing*)**

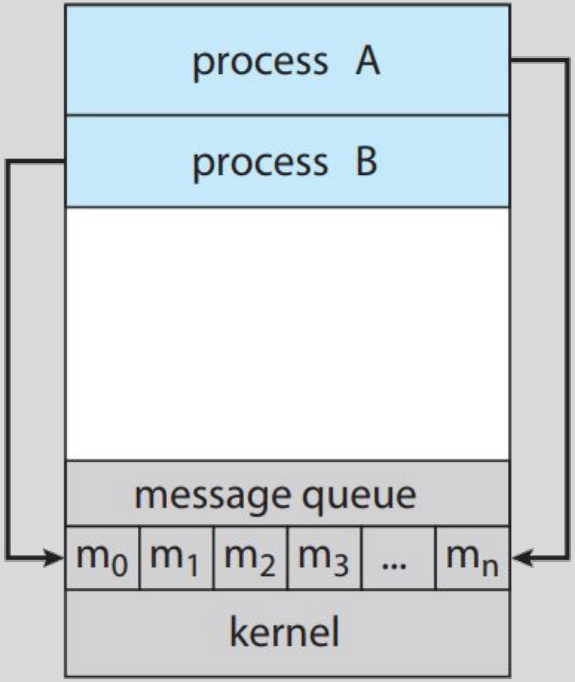
Los procesos se comunican enviando y recibiendo paquetes de datos.

- El proceso A envía un mensaje y el proceso B lo recibe a través del sistema operativo.

- **Desventaja**: Es **más lento** que la memoria compartida. Cada mensaje requiere llamar al kernel (*syscall*), lo que genera una interrupción y tiempo de gestión extra (*overhead*).



Shared memory.



Message passing.

IPC EN SISTEMAS DE MEMORIA COMPARTIDA

IPC en Sistemas de Memoria Compartida

El Mecanismo de la Memoria Compartida

Para que dos procesos compartan datos, deben romper una de las reglas de oro del Sistema Operativo: el **aislamiento de memoria**.

- **Establecimiento**: Un proceso crea un segmento de memoria en su propio espacio y los demás procesos lo "adjuntan" al suyo.
- **Permiso Especial**: El SO debe levantar la restricción que impide que un proceso lea la memoria de otro. Una vez otorgado el permiso, el SO se retira.
- **Responsabilidad**: A partir de aquí, el SO ya no supervisa qué se escribe ni dónde. Los procesos son los **únicos responsables** de no sobrescribir los datos del otro al mismo tiempo.

IPC en Sistemas de Memoria Compartida

El Problema del Productor-Consumidor

Es el ejemplo clásico para entender la cooperación. Imaginen dos roles:

- **Productor**: Genera información (como un proceso que lee datos de un sensor).
- **Consumidor**: Utiliza esa información (como un proceso que guarda esos datos en una base de datos).

Para que ambos trabajen a su propio ritmo sin chocar, necesitan un **Búfer** (un almacén temporal) ubicado en la memoria compartida.

IPC en Sistemas de Memoria Compartida

Tipos de Búfer y Sincronización

La clave es que ambos procesos deben estar sincronizados para evitar errores (por ejemplo, que el consumidor intente leer algo que aún no existe).

Existen dos estrategias según el tamaño del buffer:

- **Buffer ilimitado**: No tiene un tamaño máximo práctico. El productor siempre puede seguir fabricando elementos, pero el consumidor debe esperar si el almacén se queda vacío.

- **Buffer acotado** (Tamaño Fijo): Es el más realista.
  - Si el búfer está **lleno**, el productor debe esperar.
  - Si el búfer está **vacío**, el consumidor debe esperar.

IPC EN SISTEMAS CON PASAJE DE MENSAJES

IPC en Sistemas con Pasaje de Mensajes

El Concepto: Comunicación sin Compartir Recursos

A diferencia de la memoria compartida, aquí los procesos no comparten el espacio de memoria del otro. Se comunican mediante un **enlace** (*link*) y dos operaciones básicas: `send(message)` y `receive(message)`. Es la base de la computación distribuida (redes).

Métodos de Comunicación:

- **Comunicación Directa**: Los procesos deben conocer sus nombres (PIDs).
  - Ejemplo: `send(Proceso_B, mensaje)`.
  - **Propiedad**: El enlace es exclusivo entre dos procesos y se crea automáticamente en cuanto se conocen.
- **Comunicación Indirecta**: Los mensajes van a un "tercero", a un **Buzón** o **Puerto**.
  - Ejemplo: `send(Buzón_A, mensaje)`.
  - Propiedad: Un solo buzón puede ser compartido por muchos procesos, permitiendo que varios lean o escriban en el mismo canal.

IPC en Sistemas con Pasaje de Mensajes

Sincronización: ¿Esperar o Seguir?

Dependiendo de cómo se implementen las llamadas, el proceso puede comportarse de dos formas:

- **Síncrono** (Bloqueante): El emisor envía el mensaje y se detiene hasta que el receptor confirma su recepción, o el receptor se queda "congelado" hasta que llegue algo. Se puede pensar como una llamada telefónica.

- **Asíncrono** (No bloqueante): El emisor envía el mensaje y continúa con su ejecución. El receptor intenta leer y, si no hay nada, recibe un "nulo" y continúa. Se puede pensar como enviar un correo electrónico.

IPC en Sistemas con Pasaje de Mensajes

El Buffer: La Cola de Mensajes

Los mensajes esperan en una cola temporal antes de ser procesados. La capacidad de esta cola define el comportamiento del sistema:

1. **Capacidad Cero**: No hay almacén. El emisor **siempre** debe esperar a que el receptor recoja el mensaje en mano (esto se llama *rendezvous*).
2. **Capacidad Limitada** (N): La cola tiene un tamaño fijo. El emisor solo se bloquea si la cola está llena.
3. **Capacidad Ilimitada**: El emisor nunca se bloquea porque siempre hay sitio para dejar el mensaje.

COMUNICACIÓN EN SISTEMAS CLIENTE-SERVIDOR

Sockets

¿Qué es un Socket?

Un socket es un ***endpoint*** de comunicación. Para que dos procesos se hablen a través de una red, necesitan **un par** de sockets (uno en cada extremo).
Se identifica mediante la combinación de:

Dirección IP + Número de Puerto (Ejemplo: `192.168.1.10:80`)

Sockets

La Arquitectura Cliente-Servidor

La comunicación sigue un flujo organizado donde cada parte tiene un rol:

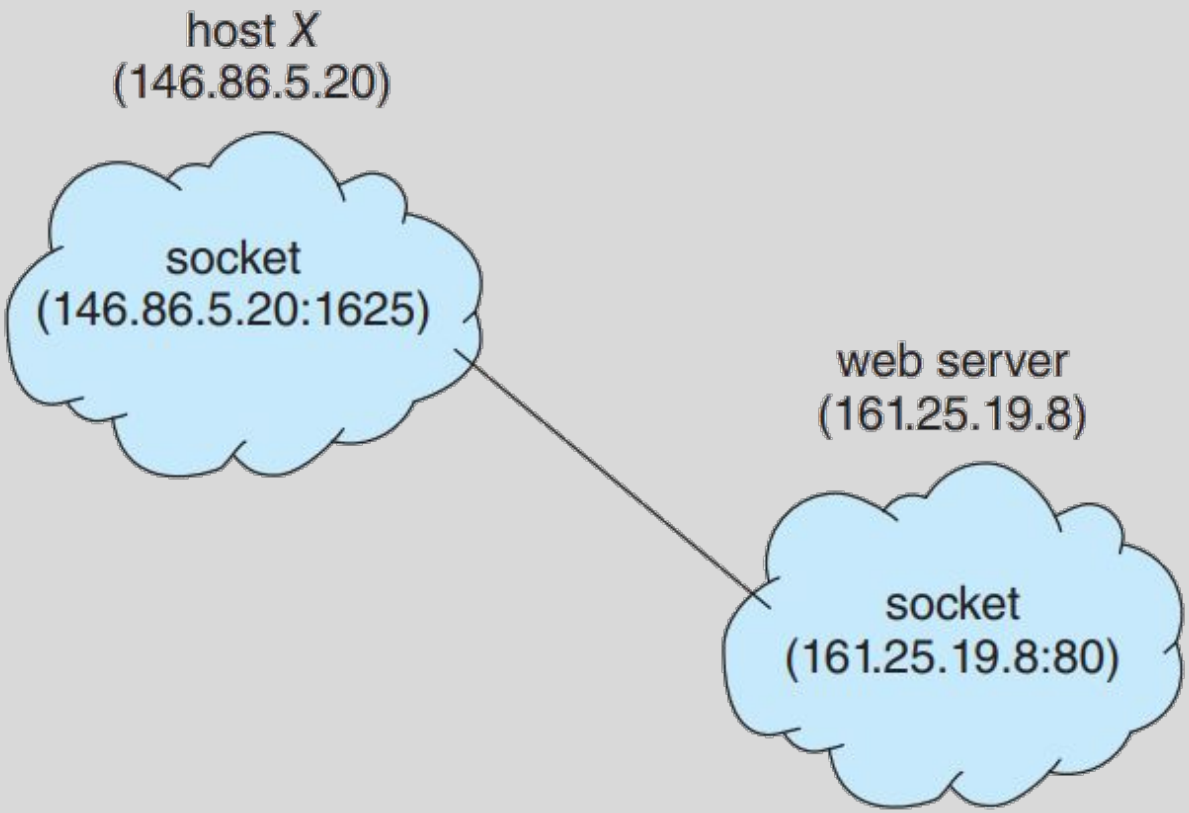
- **El Servidor** (La parte que escucha): Espera solicitudes en un puerto específico. Existen varios **puertos conocidos** (*well-known ports*, menores a 1024) para servicios estándar:
  - 22: SSH (Acceso remoto seguro).
  - 80: HTTP (Web).
  - 443: HTTPS (Web segura).

- **El Cliente** (La que envía una solicitud): Cuando quiere conectar, su sistema operativo le asigna un **puerto arbitrario** (mayor a 1024) para esa sesión específica.

Sockets

Un Par de Sockets, una Conexión Única.

Para que los paquetes de datos no se pierdan, cada conexión en Internet debe ser **única**. Esto se logra mediante un "par de sockets".

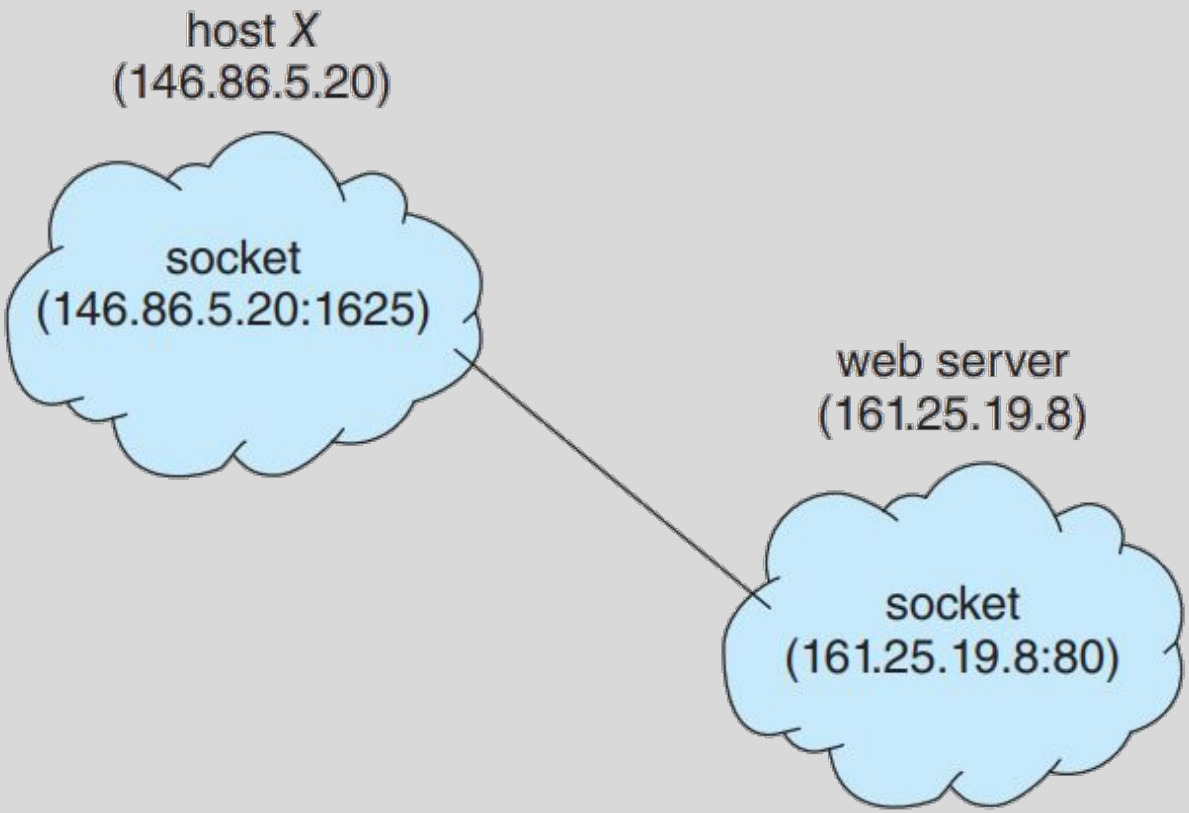


Communication using sockets.

Sockets

Ejemplo de una conexión web:

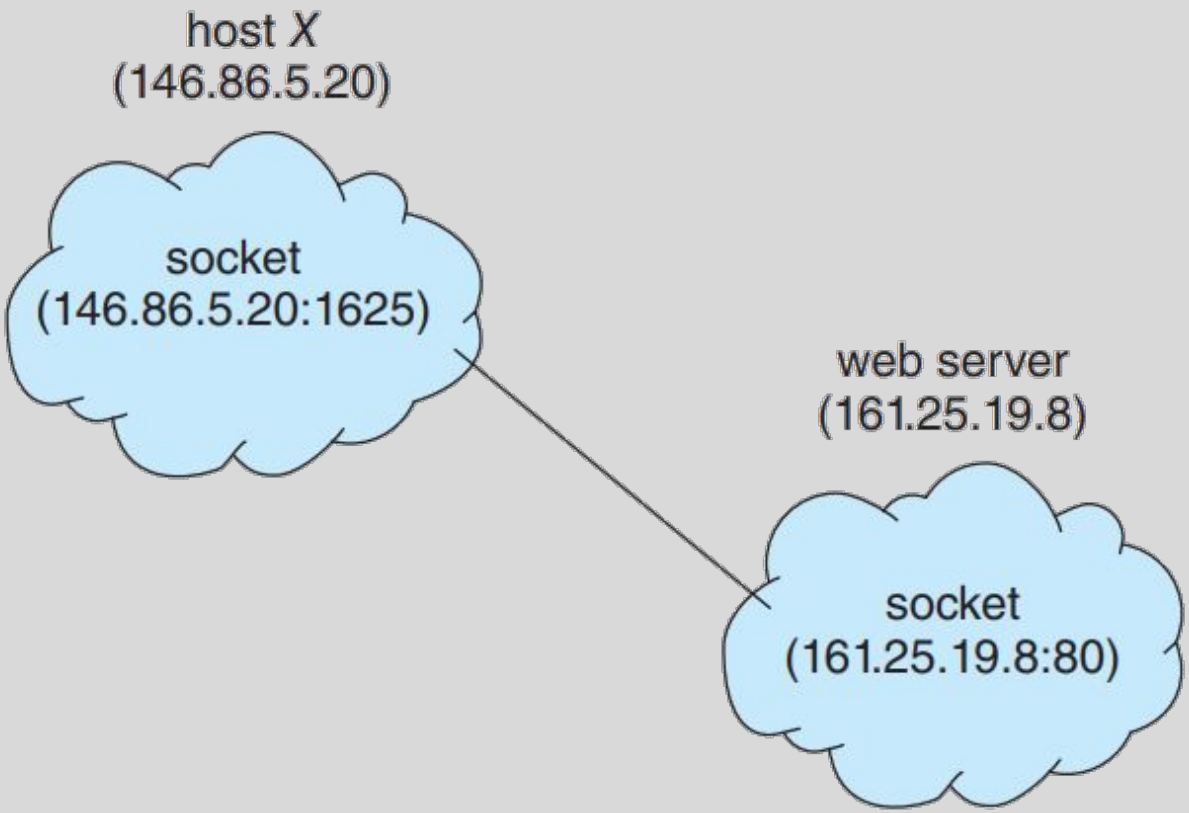
1. **Host Cliente** (IP 146.86.5.20): Abre el puerto 1625. Por lo tanto su socket es `146.86.5.20:1625`.
2. **Servidor Web** (IP 161.25.18.8): Escucha activamente en el puerto 80. Por lo tanto su socket es `161.25.19.8:80`.
3. **La Conexión** se define por el par `(146.86.5.20:1625, 161.25.19.8:80)`.



Communication using sockets.

Sockets

Si el mismo cliente abre otra pestaña del navegador hacia el mismo servidor, el sistema le asignará un puerto distinto (ej. 1626). Así, el nuevo par de sockets `(146.86.5.20:1626, 161.25.19.8:80)` es diferente al anterior, y el sistema operativo sabe exactamente a qué pestaña enviar cada dato.



Communication using sockets.

Muchas Gracias

Jeremías Fassi

Javier E. Kinter